

Clusterização, Busca Textual e Ranking

Eduardo Adame

Ciência de Dados

22 de maio de 2026





O que é clusterização?

- Agrupar pontos **sem rótulos** — o algoritmo descobre a estrutura
- Objetivo: maximizar similaridade *dentro* do grupo e minimizar *entre* grupos
- Aplicações: segmentação de clientes, detecção de anomalias, **churn**, compressão de dados, pré-processamento para classificação

Famílias de algoritmos

- **Centróides**: K-Means, K-Medoids
- **Densidade**: DBSCAN, OPTICS, HDBSCAN
- **Hierárquicos**: aglomerativo, divisivo
- **Probabilísticos**: GMM, LDA

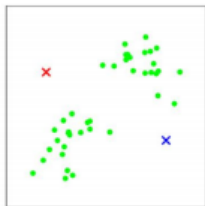
Churn — um caso de uso real

- Clientes com comportamento similar ao de quem cancelou são agrupados
- Cluster de **alto risco**: baixa frequência, tickets altos, sem renovação recente
- Cluster de **baixo risco**: uso crescente, suporte esporádico
- Permite agir *antes* do cancelamento — retenção proativa

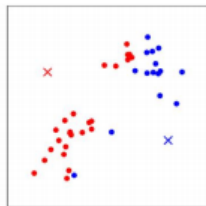
K-Means — o algoritmo



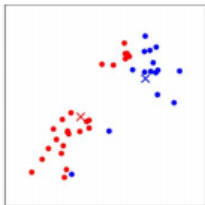
(a)



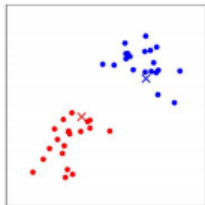
(b)



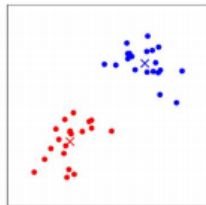
(c)



(d)



(e)



(f)

Listing 1: scikit-learn: K-Means e escolha de k

```
1 from sklearn.cluster import KMeans
2 from sklearn.preprocessing import StandardScaler
3 import matplotlib.pyplot as plt
4 # Sempre escalar antes de clusterizar
5 X_std = StandardScaler().fit_transform(X)
6 # Método do cotovelo - escolher k
7 inercias = []
8 for k in range(1, 11):
9     km = KMeans(n_clusters=k, random_state=42, n_init=10)
10    km.fit(X_std)
11    inercias.append(km.inertia_)
12 plt.plot(range(1, 11), inercias, marker="o")
13 plt.xlabel("k"); plt.ylabel("Inércia"); plt.title("Método do Cotovelo")
```

- `inertia_` = soma das distâncias quadráticas ao centróide
- **Método do cotovelo**: escolha o k onde a queda desacelera
- `n_init=10` roda 10 inicializações aleatórias — usa a melhor



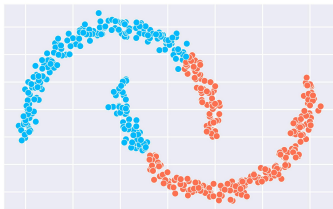
Listing 2: Silhouette score e redução dimensional

```
1 from sklearn.metrics import silhouette_score
2 from sklearn.decomposition import PCA
3 km = KMeans(n_clusters=3, random_state=42, n_init=10)
4 labels = km.fit_predict(X_std)
5 # Silhouette: [-1, 1] - quanto maior, mais coeso e separado
6 score = silhouette_score(X_std, labels)
7 print(f"Silhouette: {score:.3f}")
8 # Visualizar em 2D com PCA
9 pca = PCA(n_components=2)
10 X2d = pca.fit_transform(X_std)
11 plt.scatter(X2d[:, 0], X2d[:, 1], c=labels, cmap="tab10", alpha=0.6)
12 plt.scatter(pca.transform(km.cluster_centers_)[:, 0],
13             pca.transform(km.cluster_centers_)[:, 1],
14             marker="X", s=200, c="black", label="Centróides")
15 plt.legend()
```

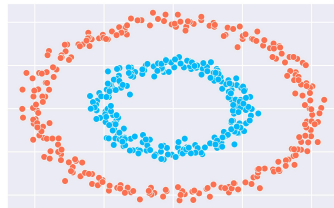
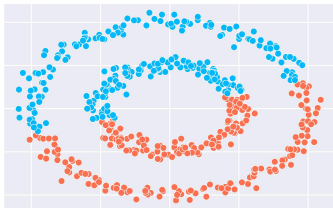
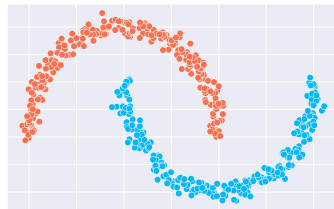
DBSCAN — clusterização por densidade



KMeans



DBSCAN



Listing 3: scikit-learn: DBSCAN e HDBSCAN

```
1 from sklearn.cluster import DBSCAN
2 # eps: raio da vizinhança; min_samples: mínimo para ser núcleo
3 db = DBSCAN(eps=0.5, min_samples=5)
4 labels = db.fit_predict(X_std)
5 n_clusters = len(set(labels)) - (1 if -1 in labels else 0)
6 n_ruido = (labels == -1).sum()
7 print(f"{n_clusters} clusters, {n_ruido} pontos de ruído")
8 # HDBSCAN - versão hierárquica, não precisa de eps
9 # pip install hdbscan (ou sklearn >= 1.3)
10 from sklearn.cluster import HDBSCAN
11 hdb = HDBSCAN(min_cluster_size=10)
12 labels_h = hdb.fit_predict(X_std)
```

- -1 = ruído — ponto que não pertence a nenhum cluster
- DBSCAN encontra clusters de *forma arbitrária* e detecta outliers
- HDBSCAN dispensa ε — mais robusto para dados reais



Pipeline de pré-processamento

1. **Tokenização** — dividir em tokens (palavras, subpalavras)
2. **Normalização** — minúsculas, remoção de acentos
3. **Stopwords** — remover palavras muito frequentes e pouco informativas
4. **Stemming / Lematização** — reduzir à raiz morfológica
5. **Vetorização** — converter tokens em números

Representações vetoriais

- **Bag of Words (BoW)**: contagem de ocorrências — simples, sem contexto
- **TF-IDF**: pondera pela raridade no corpus — muito mais discriminativo
- **Word2Vec / GloVe**: vetores densos com semântica
- **Sentence Transformers**: embeddings de sentenças inteiras

Para busca e ranking: TF-IDF. Para semântica e similaridade: embeddings.

TF-IDF — a intuição



Index →	0	1	2	3	4	5	6	7	8
	and	document	first	is	one	second	the	third	this
"This is the first document."	0	0.46979139	0.58028582	0.38408524	0	0	0.38408524	0	0.38408524
"This document is the second document."	0	0.6876236	0	0.28108867	0	0.53864762	0.28108867	0	0.28108867
"And this is the third one."	0.51184851	0	0	0.26710379	0.51184851	0	0.26710379	0.51184851	0.26710379
"Is this the first document?"	0	0.46979139	0.58028582	0.38408524	0	0	0.38408524	0	0.38408524

Listing 4: scikit-learn: vetorização e busca por cosseno

```
1 from sklearn.feature_extraction.text import TfidfVectorizer
2 from sklearn.metrics.pairwise import cosine_similarity
3 corpus = ["gato comeu peixe dormiu",
4           "cachorro correu parque latiu",
5           "gato cachorro são animais domésticos"]
6 vec = TfidfVectorizer(max_features=1000, sublinear_tf=True)
7 X = vec.fit_transform(corpus)      # matriz esparsa (docs x termos)
8 # Busca: encontrar documento mais similar à query
9 query = "animal de estimação"
10 q_vec = vec.transform([query])
11 scores = cosine_similarity(q_vec, X).flatten()
12 for i, score in sorted(enumerate(scores), key=lambda x: -x[1]):
13     print(f"[{score:.3f}] {corpus[i]}")
```

- Similaridade por cosseno: independe do comprimento do documento
- sublinear_tf=True: usa $1 + \log(\text{tf})$ — suaviza termos muito frequentes



Listing 5: SQLite + TF-IDF: buscador persistente

```
1  # 1. Armazenar documentos no banco
2  conn = sqlite3.connect("buscador.db")
3  conn.execute("CREATE TABLE IF NOT EXISTS docs "
4               "(id INTEGER PRIMARY KEY, titulo TEXT, conteudo TEXT)")
5  conn.executemany("INSERT INTO docs (titulo, conteudo) VALUES (?,?)", docs)
6  conn.commit()
7  # 2. Treinar vetorizador e serializar
8  docs_texto = [row[0] for row in conn.execute("SELECT conteudo FROM docs")]
9  vec = TfidfVectorizer(max_features=5000, sublinear_tf=True)
10 X = vec.fit_transform(docs_texto)
11 # 3. Buscar
12 def buscar(query, top_k=5):
13     vec_q = vec.transform([query])
14     scores = cosine_similarity(vec_q, X).flatten()
15     top = scores.argsort()[::-1][:top_k]
16     return conn.execute(
17         f"SELECT titulo FROM docs WHERE id IN ({','.join('?'*top_k)}") ,
18         (top + 1).tolist()).fetchall()
```

A história do PageRank

O problema de 1998

- Alta Visa e outros buscadores ranqueavam por **correspondência de palavras**
- Fácil de manipular: repetir a palavra-chave mil vezes
- Sergey Brin e Larry Page (Stanford) perguntaram: “**e se um link fosse um voto?**”
- Insight: um voto de uma página importante vale mais que de uma irrelevante

A metáfora do internauta aleatório

- Imagine um usuário seguindo links **aleatoriamente**

Definição formal

Seja N o número de páginas e $L(j)$ o conjunto de páginas que apontam para j :

$$PR(j) = \frac{1 - \alpha}{N} + \alpha \sum_{i \in L(j)} \frac{PR(i)}{|C(i)|}$$

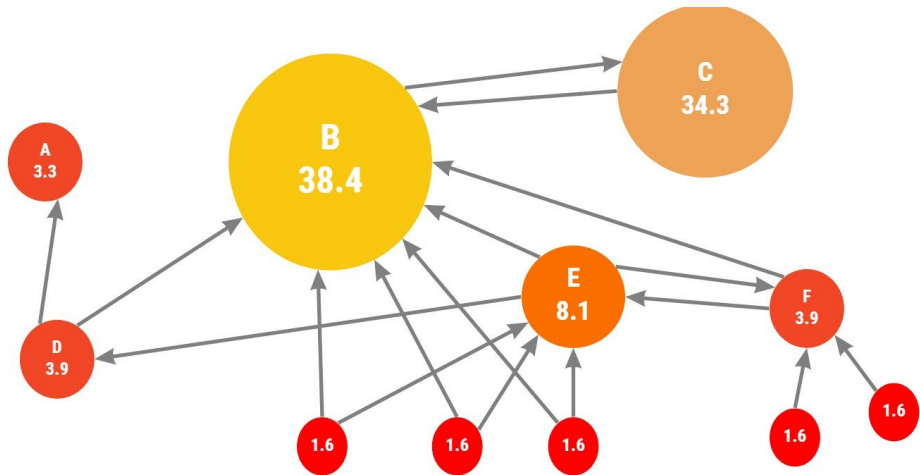
onde $|C(i)|$ é o número de links saindo de i e $\alpha \approx 0,85$.

Convergência

A iteração converge pois a matriz de transição é estocástica e irreduzível graças ao fator de teleporte.



PageRank — o internauta aleatório



miro



Listing 6: NetworkX: PageRank em grafo da Wikipedia

```
1 import networkx as nx
2 import requests
3 from bs4 import BeautifulSoup
4 def links_da_pagina(titulo):
5     url = f"https://pt.wikipedia.org/wiki/{titulo}"
6     soup = BeautifulSoup(requests.get(url,
7     headers={"User-Agent": "Mozilla/5.0"}).text, "lxml")
8     return [a["href"].replace("/wiki/", "")
9             for a in soup.select("div.mw-parser-output a[href~='/wiki/']")
10            if ":" not in a["href"]] # exclui namespaces especiais
11 G = nx.DiGraph()
12 visitados = set()
13 fila = ["Python_(linguagem_de_programação)"]
```



Listing 7: NetworkX: PageRank em grafo da Wikipedia

```
1 while fila and len(visitados) < 30:    # limitar para demo
2     pagina = fila.pop(0)
3     if pagina in visitados: continue
4     visitados.add(pagina)
5     for dest in links_da_pagina(pagina)[:10]:
6         G.add_edge(pagina, dest)
7         fila.append(dest)
8 pr = nx.pagerank(G, alpha=0.85, max_iter=100)
9 top5 = sorted(pr, key=pr.get, reverse=True)[:5]
```



Listing 8: Iteração de potências — entendendo o algoritmo

```
1 # Teleporte: internauta pula para página aleatória com prob. (1-alpha)
2 r = np.ones(N) / N
3 for _ in range(max_iter):
4     r_novo = alpha * M @ r + (1 - alpha) / N
5     if np.linalg.norm(r_novo - r, 1) < tol:
6         break
7     r = r_novo
8 return dict(zip(nós, r))
```




Listing 9: Iteração de potências — entendendo o algoritmo

```
1 # Teleporte: internauta pula para página aleatória com prob. (1-alpha)
2 r = np.ones(N) / N
3 for _ in range(max_iter):
4     r_novo = alpha * M @ r + (1 - alpha) / N
5     if np.linalg.norm(r_novo - r, 1) < tol:
6         break
7     r = r_novo
8 return dict(zip(nós, r))
```



A intuição

- Um documento é uma **mistura de tópicos**
- Cada tópico é uma **distribuição sobre palavras**
- Exemplo: documento sobre esporte pode ser 70% tópico “futebol”, 30% tópico “saúde”
- LDA *infere* os tópicos latentes a partir das co-ocorrências de palavras

Processo gerador (modelo)

1. Para cada tópico k : sorteia distribuição sobre palavras $\phi_k \sim \text{Dir}(\beta)$
2. Para cada documento d : sorteia mistura de tópicos $\theta_d \sim \text{Dir}(\alpha)$
3. Para cada palavra w em d : sorteia tópico $z \sim \theta_d$, depois palavra $w \sim \phi_z$

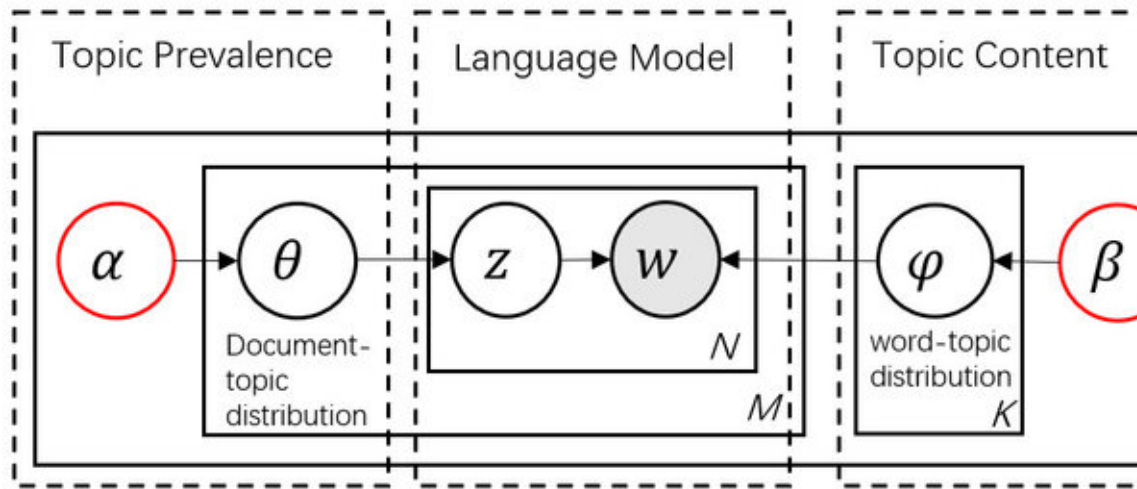


O que Dirichlet tem a ver?

- Distribuição de Dirichlet é uma **distribuição sobre distribuições**
- Parâmetro α pequeno $\Rightarrow$ documentos com poucos tópicos dominantes
- Parâmetro α grande $\Rightarrow$ mistura uniforme de tópicos
- Análogo para β e as palavras nos tópicos

Inferência

Observamos as palavras; queremos inferir θ e ϕ . Usamos Gibbs Sampling ou Variational Bayes.



Listing 10: scikit-learn: LDA em corpus de notícias

```
1 from sklearn.decomposition import LatentDirichletAllocation
2 from sklearn.feature_extraction.text import CountVectorizer
3 =
4 # LDA usa contagens (não TF-IDF)
5 vec = CountVectorizer(max_df=0.95, min_df=2,
6                       max_features=1000, stop_words="english")
7 X_counts = vec.fit_transform(corpus)
8 lda = LatentDirichletAllocation(n_components=5,      # número de tópicos
9                                max_iter=20,
10                               random_state=42)
11 lda.fit(X_counts)
12 # Palavras mais relevantes por tópico
13 termos = vec.get_feature_names_out()
14 for t, componente in enumerate(lda.components_):
15     top_idx = componente.argsort()[::-1][:8]
16     print(f"Tópico {t}: {' | '.join(termos[top_idx])}")
17 doc_topicos = lda.transform(X_counts) # shape: (n_docs, n_topics)
```

De TF-IDF a embeddings — busca semântica



Limitações do TF-IDF

- Baseado em **correspondência exata** de tokens
- “automóvel” e “carro” são completamente distintos
- Sem noção de contexto ou sinônimos
- Funciona bem para recuperação léxica; mal para semântica

Sentence Transformers

- Modelo BERT ajustado para produzir **embeddings de sentenças**
- Documentos similares ficam próximos no espaço vetorial
- Busca = encontrar vizinhos mais próximos no espaço de embeddings

Pipeline completo

1. Gerar embeddings dos documentos com **sentence-transformers**
2. Armazenar no banco (PostgreSQL com pgvector ou SQLite com sqlite-vec)
3. Na busca: gerar embedding da query e fazer busca por vizinhos mais próximos (**ANN**)
4. Retornar documentos ordenados por similaridade de cosseno



Listing 11: Embeddings + SQLite: buscador semântico

```
1 from sentence_transformers import SentenceTransformer
2 import numpy as np, sqlite3, json
3
4 model = SentenceTransformer("paraphrase-multilingual-MiniLM-L12-v2")
5
6 # Indexar documentos
7 conn = sqlite3.connect("semantico.db")
8 conn.execute("CREATE TABLE IF NOT EXISTS docs "
9              "(id INTEGER PRIMARY KEY, titulo TEXT, "
10              " conteudo TEXT, embedding TEXT)")
11
12 for titulo, texto in documentos:
13     emb = model.encode(texto).tolist()
14     conn.execute("INSERT INTO docs (titulo, conteudo, embedding) VALUES (?, ?, ?)",
15                 (titulo, texto, json.dumps(emb)))
16 conn.commit()
```



Listing 12: Embeddings + SQLite: buscador semântico

```
1 # Buscar
2 def buscar_semantico(query, top_k=5):
3     q_emb = model.encode(query)
4     rows = conn.execute("SELECT id, titulo, embedding FROM docs").fetchall()
5     scores = [(r[0], r[1],
6                 float(np.dot(q_emb, json.loads(r[2])) /
7                 (np.linalg.norm(q_emb) * np.linalg.norm(json.loads(r[2])))))
8               for r in rows]
9     return sorted(scores, key=lambda x: -x[2])[:top_k]
```


Obrigado!

Alguma dúvida?

Agora vamos para os exercícios!